

Ice-Breaker Multiplayer Game — Comprehensive Technical Report (Interview Prep)

Scope / grounding rule: Every claim below is grounded in the repository source code that was present at report-generation time. Where the code does *not* implement something (e.g., WebSockets), the report states that fact.

0. What this project does (quick orientation)

This is a multiplayer “ice-breaker” web app built on **Next.js (App Router)**. A user authenticates with **Google OAuth** (via NextAuth), fills in an avatar/profile, and then joins an **ice-breaker game session** identified by a short **game code**.

Gameplay is not based on real-time WebSockets. Instead, **clients poll HTTP endpoints** to synchronize the user’s assignment state.

1. Project Overview

1.1 First-year onboarding problem it solves

The app’s purpose is to help new participants quickly learn about each other using a guided “ice-breaker” activity:

- Users create/choose an avatar (via `avataaars`).
- Users fill an “about me” profile (`AboutUser`).
- A game session assigns each user to “assigned users” (someone they will interact with).
- Each assignment requires submitting a **selfie** that becomes part of the final collage.
- Scores are computed based on how quickly a user completes assignments.

This is implemented through the database models and game endpoints:

- `src/database/AboutUser.ts`
- `src/database/Avatar.ts`
- `src/database/Assigned.ts`
- `src/database/Selfie.ts`
- `src/app/api/user/game/[code]/route.ts` (assignment assignment/polling + selfie submission)
- `src/app/game/[code]/ended/*` (results UI)

1.2 Actual game mechanics (from code)

Core loop:

1. Game code exists (created by admin).

- `POST /api/gamecode/create` creates or returns the active code.
- Admin-only via `ADMINS` list.

2. Game waiting / started

- Each participant loads `/game/[code]` .
- UI polls `GET /api/user/game/[code]?assignedId=...` once per second.

3. Assignment lifecycle (server-side)

- If `gameCode.startedAt` is `null`: return `gameState = "waiting"` .
- When started:
 - Server calls `fetchLatestAssigned(gameCode.id, userId)` .
 - If no active assignment exists, it calls `createUserAssignment(gameCode, userId)` .
 - Returns `gameState = "started"` and the latest assigned record.

4. Submitting selfie (player action)

- `POST /api/user/game/[code]`
- Validates:
 - `selfie` is an image
 - `assignedId` belongs to the authenticated user
 - posted name matches the assigned user's `AboutUser.name`
- Updates the assignment: `assigned.update({ completedAt: new Date() })` .
- Creates a `Selfie` row containing the uploaded file bytes.

5. Completion / ending

- When the client polls and the assignment is "done" for the user, it transitions toward `completed / ended` .
- Results page: `/game/[code]/ended/...`
 - Uses `getStatsForUser(gameCode, userId)` .

6. Final collage

- `MyCollage.tsx` renders eight selfies in a fixed grid layout and provides a download via `html2canvas` .

1.3 Why 120+ concurrent users is non-trivial here

Even though there is no WebSocket server, concurrency is still challenging because:

- The game is synchronized through **high-frequency polling**:
 - In `GamePage.tsx` :
 - `setInterval(fetchAssigned, 1000)` (every second)
 - Each poll triggers a database-backed request.
- Each poll performs multiple DB queries:
 - `GameCode.findOne` including `users`
 - `Assigned.findByPk` if `assignedId` is provided
 - If started and no valid assignment, it may call `createUserAssignment` which queries `UserGame` and `Assigned`.

This is the scalability bottleneck: **database load and query fan-out due to polling**, not WebSocket throughput.

1.4 Deployment context (academic event)

The repository includes many indicators of an academic event/orientation deployment:

- `src/constants/index.ts` contains an `ADMINS` list of SIT Pune emails.
- The UI metadata uses "SIT Ice Breaker".
- The short game codes and selfie collage suit campus onboarding.

No explicit deployment narrative appears in source, but the naming and admin email list strongly indicate academic usage.

2. Tech Stack — Full Inventory

2.1 Runtime dependencies (from `package.json`)

From `ice-breaker/package.json` :

Next/React

- `next` `^14.2.5` (App Router, SSR/CSR hybrid)
- `react` `^18.3.1`
- `react-dom` `^18.3.1`

UI libraries

- `@mui/material` `^5.16.4`
- `@mui/icons-material` `^5.16.4`
- `@mui/material-nextjs` `^5.16.4`
- `@emotion/react` `^11.11.4`
- `@emotion/styled` `^11.11.5`
- `@emotion/cache` `^11.11.0`

Auth

- `next-auth` `^5.0.0-beta.20`
- Uses Google provider: `next-auth/providers/google`

Database

- `sequelize` `6.37.3`
- `sqlite3` `^5.1.7` (SQLite dialect)
- `mysql2` `^3.11.0` (present but not used by current Sequelize config)

Game logic / utilities

- `axios` `^1.7.3` (client-side API calls)
- `zod` `^3.23.8` (request validation in admin endpoint)

Media / rendering

- `html2canvas` `^1.4.1` (collage download)
- `sharp` `^0.33.4` (dependency present; not observed directly in the read snippets)
- `react-confetti` `^6.1.0` (confetti on completion)
- `react-fast-marquee` `^1.6.5` (not verified from snippets, but dependency present)
- `react-spring` `^9.7.4`
- `embla-carousel-react` `^8.1.8`

Avatar generation

- `avataaars` `^2.0.0`

2.2 Dev dependencies (from `package.json`)

- `typescript` `^5.5.3`
- `eslint` `^8.57.0`
- `eslint-config-next` `14.2.5`
- `tailwindcss` `^3.4.6`
- `postcss` `^8.4.39`
- `@faker-js/faker` `^8.4.1` (likely for `sampleData` endpoints)

2.3 Lockfile

- `package-lock.json` exists and is lockfile version 3.
- Versions are consistent with `package.json` .

2.4 Frontend vs backend vs dev dependencies

- Frontend / React runtime: `react` , `next` , `@mui/*` , `axios` , UI/animation/media libs.
 - Backend runtime (server components and route handlers): `next-auth` , `sequelize` , `sqlite3` .
 - Dev tools: `typescript` , `eslint` , `tailwindcss` , etc.
-

3. Next.js Architecture

3.1 Router type: App Router

This is an **App Router** project because route handlers and pages are under `src/app/*` .

Evidence:

- `src/app/layout.tsx`
- `src/app/page.tsx`
- API routes under `src/app/api/.../route.ts`

3.2 SSR vs CSR vs static (based on code)

SSR / Server Components

- `src/app/layout.tsx` is server-side by default.
- `src/app/page.tsx` is an `async` server component that calls `checkAuthAndRedirect()` and then redirects.
- `src/app/game/[code]/page.tsx` is an `async` server component that verifies auth.
- Result pages like `src/app/game/[code]/ended/page.tsx` are server components that fetch stats server-side (`getStatsForUser`).

CSR / Client Components

- `GamePage.tsx` begins with `"use client"`.
- Many admin/game stats pages are also `"use client"` (polling UI).

Static assets

- `src/app/globals.css`, `src/assets/fonts/*`, and `public/*`.

3.3 Directory structure (quoting key parts)

- `src/app/api/*/route.ts` (Next route handlers)
- `src/app/game/[code]/...` (game UI)
- `src/app/auth/*` (sign-in/out UI)
- `src/app/auth/signin/...` (custom sign-in pages)

3.4 Routes and what they render

App routes

- `GET / (src/app/page.tsx)`:
 - Checks auth.
 - Redirects admins to `/gamecode`.
 - If user missing `AboutUser`, redirects to `/aboutme`.
 - If user missing `Avatar`, redirects to `/aboutme/avatar`.
 - Otherwise redirects to latest valid game code or `/gamecode`.
- `GET /aboutme/*`:
 - `src/app/aboutme/page.tsx` and sub-pages.
- `GET /gamecode`:
 - Host/admin pages under `src/app/gamecode/*`.
- `GET /game/[code]`:
 - `src/app/game/[code]/page.tsx` (auth check)
 - Renders `GamePage.tsx` which polls assignment state.
- `GET /game/[code]/ended/*`:
 - `src/app/game/[code]/ended/page.tsx` (server fetch result and renders `Ended.tsx`).
 - `/mycollage` renders `MyCollage.tsx`.
- `GET /game/[code]/stats/*`:
 - `live` stats pages.

API routes

- `src/app/api/gamecode/create/route.ts`
- `src/app/api/user/game/[code]/route.ts`
- `src/app/api/user/game/[code]/skip/[assignedId]/route.ts`
- `src/app/api/gamecode/[code]/stats/route.ts`
- plus auth and other user routes (avatar/about/hobbies).

3.5 Server Components existence and why server-side

Server-side logic is used to:

- Guard routes (`checkAuthAndRedirect()`), preventing unauthenticated access.
- Perform DB operations without exposing DB credentials to the browser.
- Compute end-of-game scoring server-side (`getStatsForUser`).

Example:

- `src/app/game/[code]/ended/page.tsx` calls DB + scoring server-side and returns JSX.

4. Real-Time Multiplayer

4.1 What technology enables real-time sync?

No WebSockets / [Socket.io](#) / SSE are implemented.

Evidence from code scans:

- Searching for `socket.io` , `WebSocket` , `EventSource` , `ws` , etc in `src` returned no relevant server initialization.
- The client sync method is **polling** via `setInterval` .

4.2 Polling-based “real-time” implementation

In `src/app/game/[code]/GamePage.tsx` :

- It repeatedly calls:
 - `axios.get( /api/user/game/${code}?assignedId=${assigned?.id} )`
- Poll period:
 - `const interval = setInterval(fetchAssigned, 1000);`

So the “real-time multiplayer” is implemented by:

- frequent HTTP GET polling,
- DB reads on each poll,
- state transitions returned as JSON (`gameState` , `assigned`).

4.3 Real-time server initialization code

None exists in the repository.

4.4 Events emitted/received with payload schema

There are **no socket events**.

Instead the JSON contracts of polling endpoints are the “event system”.

Polling GET endpoint: `GET /api/user/game/[code]?assignedId=...`

From `src/app/api/user/game/[code]/route.ts` :

Possible responses:

- If assignedId is provided and `Assigned.completedAt` is null:
 - `return NextResponse.json("VALID")`
- If user is not in game:
 - `return NextResponse.json({ gameState: "notInGame" })`
- If game ended:
 - `return NextResponse.json({ gameState: "ended" })`
- If started:
 - returns `NextResponse.json({ gameState, assigned })`
 - `assigned` includes `assignedUser` data with nested `avatar` and `aboutUser` .
- If started but assignment creation fails:
 - `return NextResponse.json("COMPLETED")`
- If waiting:
 - `return NextResponse.json({ gameState: "waiting" })`

Client handling in `GamePage.tsx` :

- `if (data === "VALID") return;`
- `if (data === "COMPLETED") { setGameState("completed"); return; }`
- `setGameState(data.gameState); setAssigned(data.assigned);`

4.5 How game state is managed server-side

Server-side state is stored in the database tables:

- `game_codes` (via `GameCode` model)
- `assigned` (via `Assigned` model)
- `selfies` (via `Selfie` model)
- `user_games` (via `UserGame` join model)
- `users` , `about_user` , `avatars` for participant metadata.

No in-memory state exists for gameplay.

4.6 How rooms/lobbies are implemented

"Room" equals a `GameCode`.

- GameCode contains:
 - code unique identifier
 - expiry
 - startedAt
 - endedAt
 - userId (admin owner)

Participants are associated with game codes through the `UserGame` through table:

- `User.belongsToMany(GameCode, { through: UserGame })`
- `GameCode.belongsToMany(User, { as: "users", through: UserGame })`

4.7 Player join/leave handling

Players do not "join a socket room"; they join logically by being included in `GameCode.users` and by having an entry in `UserGame`.

The observed join mechanics in code:

- When a player hits `GET /api/user/game/[code]`, the server checks:
 - `const userInGame = gameCode.users.find((user) => user.id === userId)`
- If not found → `gameState = "notInGame"`.

The server may create an assignment upon first started poll:

- `createUserAssignment(gameCode, userId)` creates `Assigned` row.

No explicit "leave" endpoint is observed.

4.8 Disconnect mid-game

Because there are no sockets, "disconnect" means only that polling stops.

If a user disconnects:

- Their current `Assigned` record remains with `completedAt = null`.
- There is an auto-skip mechanism in `polledExpiring.ts` which updates assignments when time passes.

Auto-skip:

- `polledExpiring(code)` finds uncompleted assignments older than `ASSIGNMENT_AUTO_SKIP_MS` and sets:
 - `isSkipped: true`
 - `completedAt: new Date()`

However:

- the code snippet shows `Assigned.update(...)` is not awaited.
 - and we do not see where `polledExpiring` is scheduled from the project structure in the snippets.
-

5. Game State Machine

5.1 Game states present

Client-side `GamePage.tsx` defines:

- `waiting`
- `started`
- `ended`
- `notInGame`
- `completed`

5.2 Server-side transitions (from polling endpoint)

The server polling endpoint is:

- `GET /api/user/game/[code]` (`src/app/api/user/game/[code]/route.ts`)

It computes state using:

- `gameCode.endedAt`
- `gameCode.startedAt`
- whether `assignedId` refers to an active (uncompleted) `Assigned` record
- whether the user is included in `gameCode.users`

State mapping (as implemented):

1. Game not found

- `if (!gameCode) return 404`

2. Ended

- `if (gameCode.endedAt) { gameState = "ended"; return { gameState } }`

3. Not in game

- `const userInGame = gameCode.users.find((user) => user.id === userId)`
- `if (!userInGame) return { gameState: "notInGame" }`

4. Waiting

- `if (!gameCode.startedAt) { gameState = "waiting"; return { gameState } }`

5. Started

- When started:
 - it tries `fetchLatestAssigned(gameCode.id, userId)` where conditions require:
 - `completedAt: null`
 - `assignedAt` ordering by most recent
 - if missing assignment, it tries `createUserAssignment(gameCode, userId)`
 - if still missing assignment after creation attempt:
 - `return NextResponse.json("COMPLETED")`
 - otherwise:
 - `return NextResponse.json({ gameState: "started", assigned })`

5.3 "COMPLETED" semantics

The client treats the literal string response `"COMPLETED"` as:

- `setGameState("completed")`

That occurs when the server fails to create a new assignment:

- `const isAssigned = !!await createUserAssignment(...)`
- If `!isAssigned` → `return NextResponse.json("COMPLETED")`

5.4 State synchronization strategy

State synchronization is achieved by:

- Client-side polling every second (see `GamePage.tsx`)
- Server returns current `gameState` and assignment snapshot
- Client updates local React state (`gameState` , `assigned`)

5.5 Race condition handling / multiple simultaneous actions

The design is inherently **request/response** based; concurrency issues are handled implicitly by DB constraints and query logic rather than explicit locking.

Important concurrency behaviors:

- **Assignment creation**
 - `createUserAssignment` queries all `Assigned` records for `(gameCodeId, userId)` and filters out already-assigned targets.
 - It then selects a random possible assignee and creates a new `Assigned` row.
 - There is no explicit transaction around "check then create".
- **Assignment completion / selfie upload**
 - `POST /api/user/game/[code]` updates the assigned row by `assigned.update({ completedAt: new Date() })` and inserts a `Selfie`.
 - There is no explicit DB constraint preventing multiple completions for the same `Assigned` record, although the validation uses `assignedId` ownership and expects the selfie to correspond to that assignment.

Because multiple polls from the same user can overlap, the client tries to avoid concurrency with a local flag:

- `let isFetching = false` inside the first effect
- `if (isFetching) return;`

However, there's no server-side lock, so simultaneous requests across browser tabs/devices could still interleave.

6. Concurrency & Scaling

6.1 How the architecture supports 120+ concurrent players

The architecture is horizontally scalable in principle because:

- it stores all game state in SQLite (in `database.sqlite`) via Sequelize
- it does not keep in-memory session state

But in practice, the implemented sync method is **HTTP polling** which can overload:

- the web server instance (many requests per second)
- the database (many DB reads)

Key scaling-relevant mechanics:

- Polling frequency: every second (`setInterval(fetchAssigned, 1000)`).
- Each poll reads `GameCode` , `Assigned` (if `assignedId`), and sometimes creates new `Assigned` .

6.2 Redis pub/sub or [Socket.io](#) adapter

No Redis, no [Socket.io](#), and no pub/sub are present.

6.3 Bottlenecks at this scale

Primary bottlenecks grounded in code:

1. Database read amplification due to polling

- `GET /api/user/game/[code]` USES `GameCode.findOne({ include: ["users"] })` .

2. Query explosion from assignment creation

- When there's no active assignment, it calls `createUserAssignment` , which reads:
 - `UserGame.findAll({ where: { gameId } })`
 - `Assigned.findAll({ where: { gameId, userId } })`

6.4 Load testing performed

No load-testing scripts, configuration, or results are present in the repository tree.

7. Database (Sequelize ORM)

7.1 Underlying database used

`src/database/sequelize.ts` configures:

- `dialect: "sqlite"`
- `storage: "database.sqlite"`

So the DB used under Sequelize is **SQLite**.

7.2 Sequelize model definitions (complete code)

Models are defined as Sequelize `init()` calls in:

- `src/database/GameCode.ts`
- `src/database/Assigned.ts`
- `src/database/UserGame.ts`
- `src/database/Selfie.ts`
- `src/database/User.ts`
- `src/database/AboutUser.ts`
- `src/database/Avatar.ts`

GameCode (`src/database/GameCode.ts`)

- Table: `game_codes`
- Columns:
 - `id` INT PK autoincrement
 - `code` STRING unique with default generator:
 - `sha1(randomBytes(8) + dateISO).slice(0,5)`
 - `expiry` default now + 24h
 - `startedAt` DATE nullable default null
 - `endedAt` DATE nullable default null

Methods:

- `static validateGameCode(code)` (checks existence and expiry)
- `endGame()` sets `endedAt = new Date()`

Assigned (`src/database/Assigned.ts`)

- Table: `assigned`
- Columns:
 - `id` INT PK autoincrement
 - `assignedAt` DATE not null
 - `completedAt` DATE nullable
 - `isSkipped` BOOLEAN not null default false

(Other foreign keys are declared in TS but not shown in `init()` in the snippet provided; associations in `relations.ts` define relationships.)

UserGame (`src/database/UserGame.ts`)

- Table: `user_games`
- Model contains join table id only (as written: only `id` init)

Associations in `relations.ts` define the `userId` and `gameCodeId` foreign keys.

Selfie (`src/database/Selfie.ts`)

- Table: `selfies`
- Columns:
 - `id` INT PK autoincrement
 - `data` BLOB("medium") not null
 - `mimeType` STRING not null

User (`src/database/User.ts`)

- Table: `users`
- Columns:
 - `id` UUID PK
 - `name` STRING
 - `email` STRING unique
 - `emailVerified` DATE
 - `image` STRING
 - `actualName` STRING

AboutUser (`src/database/AboutUser.ts`)

- Table: `about_user`
- Columns:
 - `id` INT PK autoincrement
 - `name` STRING
 - `dateOfBirth` DATE
 - `homeTown` STRING
 - hobbies/guiltyPleasures/favoriteMovies/favoriteSongs as TEXT("medium")

Avatar (`src/database/Avatar.ts`)

- Table: `avatars`
- Stores avataaars parameters (strings).

7.3 Tables/models and what they represent

- `users` : authenticated Google users (NextAuth adapter)
- `about_user` : detailed participant profile
- `avatars` : avatar appearance data
- `game_codes` : creates a game session identity and lifecycle (expiry/startedAt/endedAt)
- `user_games` : join table linking users to game code sessions
- `assigned` : assignment between a user and an assigned target within a game
- `selfies` : media uploaded as part of assignment completion

7.4 Associations defined (`src/database/relations.ts`)

From `relations.ts` , associations include:

- `User.hasMany(Assigned, { as: "assignedUsers", foreignKey: "userId" })`
- `User.hasMany(Assigned, { as: "assignedToUsers", foreignKey: "assignedUserId" })`
- `Assigned.belongsTo(User, { as: "user", foreignKey: "userId" })`
- `Assigned.belongsTo(User, { as: "assignedUser", foreignKey: "assignedUserId" })`
- `User.hasOne(AboutUser, { as: "aboutUser", foreignKey: "userId" })`
- `AboutUser.belongsTo(User, { as: "user", foreignKey: "userId" })`
- `User.hasOne(Avatar, { as: "avatar", foreignKey: "userId" })`
- `Avatar.belongsTo(User, { as: "user", foreignKey: "userId" })`
- `GameCode.hasMany(Assigned, { as: "assigned", foreignKey: "gameCodeId" })`
- `Assigned.belongsTo(GameCode, { as: "gameCode", foreignKey: "gameCodeId" })`
- `User.belongsToMany(GameCode, { as: "gameCodes", foreignKey: "userId", through: UserGame })`
- `GameCode.belongsToMany(User, { as: "users", foreignKey: "gameCodeId", through: UserGame })`
- `GameCode.belongsTo(User, { as: "admin", foreignKey: "userId" })`
- `User.hasMany(GameCode, { as: "gameCode", foreignKey: "userId" })`
- `Assigned.hasOne(Selfie, { as: "selfie", foreignKey: "assignedId" })`
- `Selfie.belongsTo(Assigned, { as: "assigned", foreignKey: "assignedId" })`

7.5 Migrations

No Sequelize migrations directory is present (repository uses `.init()` model definitions and Sequelize adapter with `synchronize: true`).

7.6 Why Sequelize over Prisma / raw SQL

This rationale is not stated in code. The project uses Sequelize because:

- NextAuth adapter is explicitly using a Sequelize adapter (`src/auth/adapter.ts`).
 - The app's auth adapter (`NextAuth({ adapter: Sequelize(sequelize, ...) })`) requires a DB-integrated ORM for `User` , `Account` , `Session` , `VerificationToken` tables.
-

8. Authentication (Google OAuth)

8.1 OAuth library used

Authentication is implemented with **NextAuth**.

Google provider is configured via:

- `next-auth/providers/google`

8.2 Implemented OAuth 2.0 Authorization Code flow (as per NextAuth config)

This app does not manually build the authorization URL; it delegates to NextAuth.

Still, we can ground the flow at the configuration level:

1. Authorization URL construction

- Not in repository code; created internally by NextAuth GoogleProvider.

2. Scopes requested

- Not explicitly configured in repository code; GoogleProvider defaults are used unless overwritten (no `authorization` / `scope` override in `src/auth/index.ts`).

3. Callback handling

- NextAuth handles callback at `src/app/api/auth/[...nextauth]/route.ts` by exporting NextAuth handlers.
- `src/app/api/auth/[...nextauth]/route.ts` :
 - `export const { GET, POST } = handlers;`

4. ID token / user profile retrieval

- NextAuth handles token exchange and profile retrieval.

5. Session creation

- NextAuth uses the Sequelize adapter defined in `src/auth/adapter.ts`.
- The adapter defines model sync and create/get/update operations for Sessions.

8.3 Quote OAuth configuration

In `src/auth/index.ts` :

- Environment checks:
 - `GOOGLE_CLIENT_ID`
 - `GOOGLE_CLIENT_SECRET`
 - `NEXTAUTH_SECRET`
- NextAuth config:
 - `providers: [ GoogleProvider({ clientId, clientSecret }) ]`
 - `secret: process.env.NEXTAUTH_SECRET`
 - `adapter: Sequelize(sequelize, { models: { User }, synchronize: true })`
 - `pages: { signIn: "/auth/signin", signOut: "/auth/signout" }`

8.4 How authenticated user is linked to game sessions

Linking is via database:

- User identity is the NextAuth `User.id` (UUID).
- Game membership is via `user_games` join table:
 - `User.belongsToMany(GameCode, through: UserGame)`
- Authorization guard `checkAuthAndRedirect()` uses NextAuth `auth()` :
 - ensures `session.user.id` exists.

Examples:

- `src/app/api/user/game/[code]/route.ts` uses `session.user?.id!` .
- `createUserAssignment(gameCode, userId)` stores `userId` and `assignedUserId` .

9. Docker Configuration

9.1 Dockerfile (full quote)

`ice-breaker/dockerfile` :

```
FROM oven/bun:latest AS base

COPY . .
RUN bun install \
    && bun run build

EXPOSE 3000/tcp
ENTRYPOINT [ "bun", "run", "start" ]
```

Note: The Dockerfile uses `bun` to install and build, even though the repo includes `package-lock.json` and `npm` scripts.

9.2 docker-compose.yml

No `docker-compose.yml` is present in the repository.

9.3 Containerized services / ports

Single container implied by Dockerfile.

- Exposes port `3000/tcp` .

9.4 Environment variables injection

No Compose shows env injection. NextAuth env and Sequelize env are required at runtime.

9.5 Multi-stage build

Dockerfile uses a single build stage (`FROM oven/bun:latest AS base`) and builds in the same stage.

10. Azure Deployment

No Azure deployment files are present:

- no GitHub Actions workflows
- no Azure-specific scripts

Therefore:

- exact Azure service name (App Service vs AKS) is not grounded in repository files.
 - load balancer sticky session configuration is not present.
-

11. Game Design Decisions

11.1 Implemented ice-breaker activities

The “ice-breaker activity” is primarily:

- Provide an avatar and “about me” profile.
- Complete assignments by submitting a selfie.
- Assemble final collage.

11.2 Questions/prompts sourcing

No trivia/prompt question bank is present in the discovered files.

The only “prompt-like” validation is:

- In selfie submission, server validates posted name:
 - `postedUserName.toLowerCase().trim() !== aboutUser?.name.toLowerCase().trim()` → throws `INVALID_NAME`.

So the “prompt” is the assigned target user’s `AboutUser.name`.

11.3 Scoring implementation

Scoring is computed in `src/app/api/controllers/getStats.ts`.

For each user:

- `score = 0`
- For each assigned assignment completed:
 - `timeTaken = completedAt - assignedAt`
 - `ratio = (ASSIGNMENT_MAX_SCORE_TIME_MS - timeTaken) / ASSIGNMENT_MAX_SCORE_TIME_MS`
 - if `ratio < 0` → `ratio = 0`
 - `score += (ratio * MAX_ASSIGNMENT_SCORE) + BASE_SCORE`

Constants from `src/constants/index.ts` :

- `ASSIGNMENT_MAX_SCORE_TIME_MS = 1000 * 60 * 5` (note: comment says "5 hour" but value is 5 minutes)
- `BASE_SCORE = 10`
- `MAX_ASSIGNMENT_SCORE = 5`

11.4 Results display at end of game

- `Ended.tsx` (server fetched result) displays:
 - user name
 - total score
 - button to view collage
- `MyCollage.tsx` shows fixed selfie grid (expects at least 8 selfies).

12. UI/UX

12.1 CSS framework used

- Tailwind is configured (`tailwind.config.ts`) and classes exist.
- Additionally, MUI is used extensively.

12.2 How game board rendered

The "board" is not a canvas game board; it is a DOM-based flow:

- `Avatar` component for assigned user avatar.
- `Hobby` component for about me.
- `GameForm` for submission.

The collage is DOM images assembled and downloadable.

12.3 How real-time state reflected without full reload

Because the app polls and updates local state:

- `GamePage.tsx` rerenders based on `gameState` and `assigned`.

No full page reload is required after initial navigation.

12.4 Animations/transitions

- Confetti on completion via `react-confetti` in `GamePage.tsx`.
 - MyCollage has `html2canvas` capture but no animation.
 - There is a `RevealResultPage.css` file for top-results reveal animations (not fully detailed because relevant usage was not read here).
-

13. Configuration & Environment

13.1 Environment variables from `.env.example`

The repo includes `ice-breaker/.env.example`, but the environment file contents could not be read due to tooling restrictions that prevent reading `.env`-suffixed files.

What we can ground from code:

- `src/auth/index.ts` requires:
 - `GOOGLE_CLIENT_ID`
 - `GOOGLE_CLIENT_SECRET`
 - `NEXTAUTH_SECRET`

13.2 Local setup steps needed

From code usage patterns:

- Create SQLite file `database.sqlite` or ensure DB is initialized via Sequelize `synchronize: true` (NextAuth adapter sync).
- Configure Google OAuth credentials.

13.3 Required secrets

- `GOOGLE_CLIENT_ID`
 - `GOOGLE_CLIENT_SECRET`
 - `NEXTAUTH_SECRET`
-

14. Interview Q&A Preparation

Below are detailed, code-grounded answers.

14.1 “How does real-time synchronization work across 120+ concurrent players?”

It doesn't use WebSockets.

Synchronization is implemented by **client polling**:

- `GamePage.tsx` sets `setInterval(fetchAssigned, 1000)`.
- Each poll calls `GET /api/user/game/[code]?assignedId=<assignedId>`.

The server:

- reads the `game_codes` record (`GameCode.findOne`) and whether the user belongs to it
- returns a JSON object with `gameState` and (when started) the current `assigned` row.

The client updates local React state and rerenders.

At 120+ users, the dominant cost is DB load due to frequent polling and query fan-out.

14.2 “Walk me through the Google OAuth flow in this application.”

At a configuration level:

- NextAuth is configured in `src/auth/index.ts`.
- It uses `GoogleProvider({ clientId, clientSecret })`.
- It uses a Sequelize adapter (via `src/auth/adapter.ts`) with `synchronize: true`.
- NextAuth route handlers are exported at `src/app/api/auth/[...nextauth]/route.ts`.

The authorization code exchange and ID token/profile retrieval are handled by NextAuth internals (not manually implemented in repo code).

After successful auth:

- `checkAuthAndRedirect.ts` calls `auth()` from NextAuth.
- It ensures `session.user.id` exists; otherwise redirects to `/auth/signin`.

14.3 “How did you deploy this with Docker on Azure? What Azure service?”

The repository contains a Dockerfile but does not include Azure deployment configurations or workflows.

So the only grounded info is:

- `dockerfile` builds and starts the Next app on port 3000.

The Azure service choice and Azure-specific WebSocket/load balancer configuration are not present in code.

14.4 “How do you handle a player who disconnects mid-game?”

No WebSockets exist; thus “disconnect” only affects polling.

The backend keeps assignment state in DB (`Assigned.completedAt` , `Assigned.isSkipped`).

There is an **auto-skip** mechanism in `polledExpiring.ts` that marks assignments older than `ASSIGNMENT_AUTO_SKIP_MS` as skipped/completed.

If the client disconnects mid-assignment:

- their `Assigned` remains uncompleted until either:
 - they reconnect and submit selfie, or
 - auto-skip triggers.

14.5 “Why did you choose Sequelize over Prisma for this project?”

A direct rationale is not documented.

Code-grounded reason:

- NextAuth adapter is explicitly implemented for Sequelize (`src/auth/adapter.ts`) and uses Sequelize models (`User` , `Account` , `Session` , `VerificationToken`) generated from `src/auth/models.ts` .

14.6 “What happens if the WebSocket server crashes during a game session?”

No WebSocket server exists in this repository.

Crash impact is therefore DB/polling dependent (not socket dependent).

14.7 “How would you scale this to 1,000 concurrent players?”

Code-grounded scaling observations:

- Polling every second causes DB amplification.

To scale to 1,000+:

- reduce poll frequency (adaptive polling or server push)
- cache `GameCode` and assignment lookups
- add DB indexes (e.g., on `Assigned(gameCodeId, userId, completedAt)` and `GameCode(code)`)
- consider moving from HTTP polling to WebSockets/SSE with state diffing (currently absent)
- replace SQLite with a network DB for multi-instance writes (since the app currently uses local `database.sqlite` storage)

14.8 “Why Next.js for a multiplayer game? What were the SSR trade-offs?”

From code:

- SSR/server components are used for auth gating and end-of-game scoring.
- Client components (`GamePage.tsx`) handle frequent state updates via polling.

Trade-offs implied by design:

- SSR improves security for protected pages and avoids client exposing DB credentials.
 - Frequent updates happen in client components, reducing SSR burden but increasing API/DB load.
-

Appendix A — Key code paths (for interview quoting)

- Auth guard: `src/utils/checkAuthAndRedirect.ts`
- NextAuth config: `src/auth/index.ts`
- NextAuth Sequelize adapter: `src/auth/adapter.ts`
- Game page polling: `src/app/game/[code]/GamePage.tsx`
- Assignment polling + submission: `src/app/api/user/game/[code]/route.ts`
- Admin game code creation: `src/app/api/gamecode/create/route.ts`
- Scoring: `src/app/api/controllers/getStats.ts`
- Models: `src/database/*.ts`
- SQLite dialect: `src/database/sequelize.ts`
- Dockerfile: `dockerfile`